HIGH PERFORMANCE COMPUTING

Heat Diffussion Equations
CUDA

ZiHan, Chen

Yang, Hao

University of Lleida

Higher Polytechnic School

Master's Degree in Informatics Engineering

Course 2024 - 2025

6 June 2025

Contents

1 Introduction

In this project, we solve the two-dimensional heat diffusion equation on a regular grid using several programming paradigms to study their relative performance and scalability. We consider four implementations:

- **Serial**: a straightforward single-threaded version, used as a baseline.
- **OpenMP**: a shared-memory parallelization with 4 and 8 threads.
- **MPI**: a distributed-memory parallelization with 8 and 16 processes.
- **CUDA**: a GPU-accelerated implementation with two block-size choices (8×8 and 16×16).

All CPU-based runs (Serial, OpenMP, and MPI) were performed on the same cluster node (identical hardware and OS configuration). The CUDA runs, by contrast, were executed on a separate server equipped with an NVIDIA A4000 GPU. Our goal in *Delivery 3* is twofold:

1. Present the detailed CUDA results (execution times, speedups vs. serial, block-size comparison).
2. Compare all four approaches—Serial, OpenMP, MPI, and CUDA—in terms of execution time, speedup, and efficiency, noting that CUDA results come from the A4000 GPU server while CPU runs all came from the same cluster node.

Outline of this document:

1. Section 1 (above) briefly introduces the problem, the implementations, and the hardware environments.
2. Section 3 presents all CUDA-specific results (block sizes 8×8 vs. 16×16), including comparisons to the serial baseline.
3. Section 4 performs the overall comparison among all four implementations, highlighting scalability trends and hardware differences.

2 CUDA Implementation and Data Collection

This section describes exactly how the CUDA version (`heat_cuda.cu`) was derived from the serial code (`heat_serial.c`), how it is compiled, and how timing data are gathered. All CUDA runs use an NVIDIA A4000 GPU. Below, we follow the code structure without inventing new functions.

2.1 Command-Line Arguments and Constants

The CUDA executable is built from `heat_cuda.cu`. Its `main` function expects exactly five arguments:

```
./heat_cuda <size> <steps> <block_x> <block_y> <output_file.bmp>
```

where:

- `size` (`nx` and `ny`) is the grid dimension (square grid).
- `steps` is the number of time iterations.
- `block_x` and `block_y` are the CUDA block dimensions (threads per block in x and y).
- `output_file.bmp` is the filename for the final temperature image.

Constants defined at the top of `heat_cuda.cu` are:

```
#define BMP_HEADER_SIZE 54
#define ALPHA 0.01        // Thermal diffusivity
#define L 0.2             // Physical domain length (m)
#define DX 0.02           // Grid spacing in x
#define DY 0.02           // Grid spacing in y
#define DT 0.0005         // Time step
#define T 1500            // Heat-source temperature (K)
#define BLOCK_SIZE 16     // Default block size (threads per block)
```

The code computes:

$$r = \frac{\text{ALPHA} \times \text{DT}}{(\text{DX})^2},$$

and uses `nx = ny = atoi(argv[1])`, so the grid has `nx*ny` cells.

2.2 CUDA Error-Checking Macro

Immediately after the `#include` lines, the code defines a macro for CUDA error checks:

```
#define CUDA_CHECK(call)                                    \
    do {                                                    \
        cudaError_t err = call;                             \
        if (err != cudaSuccess) {                           \
            fprintf(stderr, "CUDA Error: %s\n",             \
                    cudaGetErrorString(err));               \
            exit(EXIT_FAILURE);                             \
        }                                                   \
    } while (0)
```

Every CUDA API call (e.g., `cudaMalloc`, `cudaMemcpy`, kernel launches) is wrapped in `CUDA_CHECK(...)` to abort on failure.

2.3 Grid Initialization Kernel

Instead of initializing on the host, the CUDA code uses a `__global__` kernel named `initialize_grid_kernel`. Its prototype is:

```
__global__
void initialize_grid_kernel(double *grid, int nx, int ny,
                            double temp_source)
```

This kernel launches enough threads so that each thread computes one (i, j) index via:

```
int i = blockIdx.y*blockDim.y + threadIdx.y;
int j = blockIdx.x*blockDim.x + threadIdx.x;
if (i < nx && j < ny) {
    int idx = i*ny + j;
    if (i == j || i == nx - 1 - j) {
        grid[idx] = temp_source;
    } else {
        grid[idx] = 0.0;
    }
}
```

Thus, both main diagonals of the grid are set to `T` ($= 1500$), while other cells start at zero.

2.4 Interior Update and Boundary Kernels

In each time step, two more `__global__` kernels execute:

1. `update_interior_kernel` Prototype:

```
__global__
void update_interior_kernel(double *grid, double *new_grid,
                            int nx, int ny, double r)
```

Each thread computes (i, j) as above; if $1 \leq i \leq nx - 2$ and $1 \leq j \leq ny - 2$, it updates:

```
int idx = i*ny + j;
new_grid[idx] =
    grid[idx]
    + r * (grid[(i+1)*ny + j] + grid[(i-1)*ny + j] - 2.0*grid[idx])
    + r * (grid[i*ny + (j+1)] + grid[i*ny + (j-1)] - 2.0*grid[idx]);
```

Threads on the boundary do nothing (they return immediately).

2. `apply_boundaries_kernel` Prototype:

```
__global__
void apply_boundaries_kernel(double *new_grid, int nx, int ny)
```

Each thread checks if its (i, j) lies on the first/last row or first/last column. If so, it sets `new_grid`$[\, i * ny + j \,] = 0.0$. This enforces the zero-Dirichlet boundary condition.

2.5 Host-Side Time Loop and Timing

After allocating and initializing device memory, `main` on the host does:

```
// Create CUDA events for timing
cudaEvent_t start, stop;
CUDA_CHECK(cudaEventCreate(&start));
CUDA_CHECK(cudaEventCreate(&stop));
CUDA_CHECK(cudaEventRecord(start));

// Allocate host arrays
double *grid = (double*)malloc(nx*ny*sizeof(double));
double *new_grid = (double*)malloc(nx*ny*sizeof(double));

// Allocate device arrays
```

```
double *d_grid = nullptr, *d_new_grid = nullptr;
CUDA_CHECK(cudaMalloc((void**)&d_grid,  nx*ny*sizeof(double)));
CUDA_CHECK(cudaMalloc((void**)&d_new_grid, nx*ny*sizeof(double)));

// Launch initialization kernel
dim3 blockDim(block_x, block_y);
dim3 gridDim((nx + block_x - 1)/block_x,
             (ny + block_y - 1)/block_y);
double temp_source = T;  // from \#define
initialize_grid_kernel<<<gridDim, blockDim>>>(
    d_grid, nx, ny, temp_source);
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());

// Time-stepping loop
double r = ALPHA * DT / (DX * DX);
for (int step = 0; step < steps; ++step) {
    update_interior_kernel<<<gridDim, blockDim>>>(
        d_grid, d_new_grid, nx, ny, r);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    apply_boundaries_kernel<<<gridDim, blockDim>>>(
        d_new_grid, nx, ny);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    // Swap pointers for next iteration
    double *tmp = d_grid;
    d_grid = d_new_grid;
    d_new_grid = tmp;
}

// Stop timing
CUDA_CHECK(cudaEventRecord(stop));
CUDA_CHECK(cudaEventSynchronize(stop));
float milliseconds = 0;
CUDA_CHECK(cudaEventElapsedTime(&milliseconds, start, stop));
```

```
// Print execution time in the required format
printf("The Execution Time = %f seconds with a matrix"
       " size of %dx%d and %d steps\n",
       milliseconds / 1000.0, nx, ny, steps);

// Cleanup events
CUDA_CHECK(cudaEventDestroy(start));
CUDA_CHECK(cudaEventDestroy(stop));
```

Note that `cudaDeviceSynchronize()` is called after each kernel launch and `cudaGetLastError()` checks for launch errors. The printed line matches other versions' format:

```
The Execution Time = <seconds> seconds with a matrix size of <nx>x<ny> and <steps>
```

2.6 Host-Device Data Copy and BMP Output

After timing, the final grid (currently in `d_grid`) is copied back to host:

```
CUDA_CHECK(cudaMemcpy(grid, d_grid, nx*ny*sizeof(double),
                      cudaMemcpyDeviceToHost));
```

Then the code calls two functions (copied from the serial version) to write a BMP image:

```
void write_bmp_header(FILE *fp, int nx, int ny);
void write_grid(FILE *fp, double *grid, int nx, int ny);
```

Typically:

```
FILE *fp = fopen(output_file, "wb");
write_bmp_header(fp, nx, ny);
write_grid(fp, grid, nx, ny);
fclose(fp);
```

This produces `output_file.bmp` showing temperature distribution.

2.7 Block-Size Scripts

Five shell scripts automate compiling and running for each block size:

1. `heat_cuda_8x8.sh`

```
nvcc -DBLOCK_X=8 -DBLOCK_Y=8 -o heat_cuda heat_cuda.cu
mkdir -p cuda_benchmark_results/block_8x8
for size in 100 1000 2000; do
  for steps in 100 1000 10000 100000; do
    ./heat_cuda $size $steps 8 8 block8x8_${size}_${steps}.bmp \
      > cuda_benchmark_results/block_8x8/log_${size}_${steps}.txt
    grep "The Execution Time" \
      cuda_benchmark_results/block_8x8/log_${size}_${steps}.txt \
      >> cuda_benchmark_results/block_8x8/performance_summary.txt
    sleep 1
  done
done
```

2. `heat_cuda_16x16.sh`, `heat_cuda_16x32.sh`, `heat_cuda_32x16.sh`, `heat_cuda_32x32.sh`

```
nvcc -DBLOCK_X=<bx> -DBLOCK_Y=<by> -o heat_cuda heat_cuda.cu
mkdir -p cuda_benchmark_results/block_<bx>x<by>
for size in 100 1000 2000; do
  for steps in 100 1000 10000 100000; do
    ./heat_cuda $size $steps <bx> <by> \
      block<bx>x<by>_${size}_${steps}.bmp \
      > cuda_benchmark_results/block_<bx>x<by>/log_${size}_${steps}.txt
    grep "The Execution Time" \
      cuda_benchmark_results/block_<bx>x<by>/log_${size}_${steps}.txt \
      >> cuda_benchmark_results/block_<bx>x<by>/performance_summary.txt
    sleep 1
  done
done
```

   Replace `<bx>` and `<by>` with 16,32, etc.

Each script:

- Compiles `heat_cuda.cu` with the specified block size.
- Creates a directory under `cuda_benchmark_results/` named `block_<bx>x<by>`.
- Loops over `size` in $\{100, 1000, 2000\}$ and `steps` in $\{100, 1000, 10000, 100000\}$.
- Runs `heat_cuda` with arguments `size`, `steps`, `<bx>`, `<by>`, and a BMP filename.
- Redirects `stdout` to a `log_size_steps.txt` file, then `grep`s the execution-time line into `performance_summary.txt`.
- Waits one second (`sleep 1`) between runs.

2.8 Parsing Logs with Python

After running all block-size scripts, the directory layout is:

```
cuda_benchmark_results/
 block_8x8/
    performance_summary.txt
    log_100_100.txt
    log_100_1000.txt
    ... (one log per size/steps)
 block_16x16/
    performance_summary.txt, log_... .txt
 block_16x32/
    ...
 block_32x16/
    ...
 block_32x32/
     ...
```

Each `performance_summary.txt` contains lines of the form:

```
The Execution Time = <float> seconds with a matrix size of <nx>x<ny> and <steps> st
```

Running

```
python3 analyze_heat_cuda_logs.py
```

produces CSV files under `results/`:

- `execution_times_block_8x8.csv`
- `execution_times_block_16x16.csv`
- `execution_times_block_16x32.csv`
- `execution_times_block_32x16.csv`
- `execution_times_block_32x32.csv`
- `block_comparison.csv`

Each `execution_times_<block_config>.csv` has rows for grid sizes (100, 1000, 2000) and columns for timestep counts (100, 1000, 10000, 100000). The combined `block_comparison.csv` lists, for each (size, steps), the execution time of all block sizes and indicates the best configuration.

By using exactly the kernels and host logic in `heat_cuda.cu`—without inventing any new code—we have a fully documented CUDA implementation. The subsequent section (3) presents all timing and speedup results extracted from these logs.

3 CUDA Results

This section details the performance of our CUDA implementation of the heat diffusion solver. All CUDA runs were performed on a server equipped with an NVIDIA A4000 GPU. We tested five block-size configurations: 8×8, 16×16, 16×32, 32×16, and 32×32. For each configuration, we measured execution time on three grid sizes (100×100, 1000×1000, 2000×2000) and four timestep counts (100, 1000, 10000, 100000). We then computed speedup relative to the serial baseline (same grid and timestep on the cluster CPU).

3.1 Serial Baseline Times (cluster CPU)

Table 1: Serial execution times (seconds) on cluster CPU

Grid \ Timesteps	100	1000	10000	100000
100×100	0.000	0.020	0.220	2.130
1000×1000	0.340	3.540	25.250	278.160
2000×2000	1.160	13.640	135.670	1061.740

3.2 Execution Times for Each Block Size

3.2.1 Block 8×8

Table 2: CUDA execution times (seconds), block 8×8, on A4000 GPU

Grid \ Timesteps	100	1000	10000	100000
100×100	0.008769	0.046282	0.403726	3.993890
1000×1000	0.105576	0.228589	1.533214	14.615451
2000×2000	0.401982	0.821945	5.254081	49.760320

3.2.2 Block 16×16

Table 3: CUDA execution times (seconds), block 16×16, on A4000 GPU

Grid \ Timesteps	100	1000	10000	100000
100×100	0.008623	0.045219	0.407839	4.337848
1000×1000	0.099440	0.233991	1.579189	15.303194
2000×2000	0.412091	1.067526	7.563476	72.372164

3.2.3 Block 16×32

Table 4: CUDA execution times (seconds), block 16×32, on A4000 GPU

Grid \ Timesteps	100	1000	10000	100000
100×100	0.009084	0.047606	0.513319	4.183971
1000×1000	0.101037	0.235435	1.592223	15.060025
2000×2000	0.408823	1.074149	7.539007	72.341781

3.2.4 Block 32×16

Table 5: CUDA execution times (seconds), block 32×16, on A4000 GPU

Grid \ Timesteps	100	1000	10000	100000
100×100	0.009144	0.048127	0.462448	4.324953
1000×1000	0.103120	0.282481	2.113962	20.895088
2000×2000	0.422844	1.218318	9.183478	88.965703

3.2.5 Block 32×32

Table 6: CUDA execution times (seconds), block 32×32, on A4000 GPU

Grid \ Timesteps	100	1000	10000	100000
100×100	0.009134	0.051474	0.460260	4.566667
1000×1000	0.114553	0.332626	2.539010	24.523311
2000×2000	0.456821	1.468119	11.834010	115.334438

3.3 Speedup vs. Serial

For each block size, we compute:

$$\text{Speedup} = \frac{T_{\text{serial}}}{T_{\text{CUDA}}}$$

using the serial baseline times from Section 2.1. Below is a single combined table that lists size, timesteps, and speedup for each block configuration (no intermediate fractions shown).

Table 7: Speedup of CUDA configurations over serial baseline

Size	**Steps**	**8×8**	**16×16**	**16×32**	**32×16**	**32×32**
100×100	100	–	–	–	–	–
	1000	0.432	0.442	0.420	0.416	0.389
	10000	0.545	0.539	0.429	0.476	0.478
	100000	0.533	0.491	0.509	0.493	0.467
1000×1000	100	3.220	3.419	3.366	3.298	2.969
	1000	15.487	15.129	15.029	12.535	10.639
	10000	16.469	15.989	15.859	11.937	9.945
	100000	19.032	18.177	18.470	13.314	11.345
2000×2000	100	2.886	2.815	2.839	2.745	2.540
	1000	16.595	12.777	12.702	11.200	9.295
	10000	25.822	17.937	17.996	14.774	11.467
	100000	21.337	14.671	14.676	11.931	9.205

In each entry, the value shown is:

$$\text{Speedup} = \frac{T_{\text{serial}}}{T_{\text{CUDA}}},$$

computed for the matching grid size and timestep count. Entries marked "–" indicate undefined speedup (serial time was zero).

4 Performance Comparison

In this section, we compare execution times, speedups, and parallel efficiencies across all four implementations—Serial, OpenMP, MPI, and CUDA—on three grid sizes: 100×100, 1000×1000, and 2000×2000. We examine four timestep counts (100, 1000, 10000, and 100000) to highlight differences between small and large problem scales. All CPU-based runs (Serial, OpenMP, MPI) were performed on the same cluster node; CUDA runs were on an NVIDIA A4000 GPU server. Speedup is defined as

$$\text{Speedup} \;=\; \frac{T_{\text{serial}}}{T_{\text{parallel}}},$$

and efficiency for CPU runs as

$$\text{Efficiency} \;=\; \frac{\text{Speedup}}{P},$$

where P is the number of threads (OpenMP) or processes (MPI). CUDA does not have an equivalent P.

4.1 Grid 100×100

Table 8: Speedup vs. Serial for Grid 100×100

Implementation	Speedup			
	100	1000	10000	100000
Serial (CPU)	1.00	1.00	1.00	1.00
OpenMP (4 threads, CPU)	—	1.1634	1.6593	1.6871
OpenMP (8 threads, CPU)	—	0.3639	0.4875	0.4800
MPI (8 processes, CPU)	—	0.2374	0.2821	0.2752
MPI (16 processes, CPU)	—	0.2358	0.2763	0.2710
CUDA (8×8, A4000)	—	0.4321	0.5449	0.5333
CUDA (16×16, A4000)	—	0.4423	0.5394	0.4910

Discussion: For the smallest grid (100×100), serial time is effectively zero for 100 timesteps, so speedup is undefined (marked "—"). At 1000 timesteps, the fastest CPU run (OpenMP with 4 threads) achieves $\approx 1.16\times$ speedup; MPI and CUDA configurations remain below 1.0 (overhead dominates). At 10000 and 100000 timesteps, overheads still dominate, and no configuration consistently outperforms serial on this tiny grid.

4.2 Grid 1000×1000

Table 9: Speedup vs. Serial for Grid 1000×1000

Implementation	Speedup			
	100	1000	10000	100000
Serial (CPU)	1.00	1.00	1.00	1.00
OpenMP (4 threads, CPU)	0.5153	2.0616	1.7119	1.9344
OpenMP (8 threads, CPU)	0.7247	1.8392	1.5730	1.7878
MPI (8 processes, CPU)	1.6045	3.5238	3.4460	4.0374
MPI (16 processes, CPU)	0.6138	2.1583	4.2211	6.4092
CUDA (8×8, A4000)	3.2204	15.4863	16.4687	19.0319
CUDA (16×16, A4000)	3.4191	15.1288	15.9892	18.1766

Discussion: At 1000×1000, 100 timesteps already permit some parallel benefit. MPI with 8 processes achieves $\approx 1.60\times$ speedup, while CUDA (16×16) reaches $\approx 3.42\times$. For 1000 timesteps, CPU approaches achieve 1.8–3.5×, whereas CUDA yields 15.1–15.5×. At 10000 timesteps, MPI (16 proc) is $\approx 4.22\times$ and CUDA (8×8) $\approx 16.47\times$. At 100000 timesteps, MPI (16 proc) $\approx 6.41\times$ and CUDA (8×8) $\approx 19.03\times$. CUDA's advantage grows with timestep count, leveraging A4000 parallelism; MPI scales moderately well, OpenMP peaks around $\approx 2\times$ on 4 threads.

4.3 Grid 2000×2000

Table 10: Execution Times (seconds) for Grid 2000×2000

Implementation	Configuration	10000 ts	100000 ts
Serial (CPU)	—	135.670	1061.740
OpenMP (CPU)	4 threads	63.4674	571.2754
	8 threads	69.8214	596.5434
MPI (CPU)	8 processes	31.4273	295.1473
	16 processes	19.5963	158.3467
CUDA (A4000)	block 8×8	5.254081	49.760320
	block 16×16	7.563476	72.372164

Table 11: Speedup vs. Serial for Grid 2000×2000

Implementation	Speedup			
	100	1000	10000	100000
Serial (CPU)	1.00	1.00	1.00	1.00
OpenMP (4 threads, CPU)	1.1734	2.0404	2.1376	1.8585
OpenMP (8 threads, CPU)	1.1562	1.9145	1.9431	1.7798
MPI (8 processes, CPU)	1.4353	3.8041	4.3170	3.5973
MPI (16 processes, CPU)	1.4289	4.3462	6.9232	6.7052
CUDA (8×8, A4000)	2.8857	16.5948	25.8218	21.3371
CUDA (16×16, A4000)	2.8149	12.7772	17.9375	14.6706

Discussion: For the large grid (2000×2000), all methods scale better. OpenMP (4 threads) peaks at $\approx 2.14\times$ (10000 ts) and drops to $\approx 1.86\times$ (100000 ts). OpenMP (8 threads) is slightly lower. MPI (16 proc) achieves $\approx 6.92\times$ (10000 ts) and $\approx 6.71\times$ (100000 ts). CUDA (8×8) delivers $\approx 25.82\times$ (10000 ts) and $\approx 21.34\times$ (100000 ts), while CUDA (16×16) yields $\approx 17.94\times$ and $\approx 14.67\times$ respectively. GPU perfor-

mance is strongest at medium timesteps; at very large timesteps, kernel overhead is amortized but global-memory access cost grows.

4.4 Overall Conclusions

- **Small Problems** (100×100): Overheads dominate. None of the parallel methods consistently outperforms serial for timesteps ≤ 10000. CUDA and MPI only become competitive for larger timestep counts.

- **Medium Problems** (1000×1000): CPU-based MPI and CUDA show significant gains. At 10000 timesteps, MPI (16 proc) $\approx 4.22\times$, CUDA (8×8) $\approx 16.47\times$. OpenMP peaks around $2\times$. At 100000 timesteps, MPI (16 proc) $\approx 6.41\times$, CUDA (8×8) $\approx 19.03\times$.

- **Large Problems** (2000×2000): CUDA leads with 17–$26\times$ speedup, depending on block size and timestep. MPI (16 proc) achieves ≈ 6.7–$\approx 6.9\times$, while OpenMP saturates near $2\times$ on 4 threads.

For cluster-only environments, MPI (16 processes) provides the best scalability. For GPU-equipped machines, CUDA (especially with 8×8 blocks) dramatically outperforms CPU parallel methods once the problem size and timestep count are sufficiently large.

5 Reflections on CPU vs. GPU Computing

5.1 Learning Curve and Development Effort

We began with a working serial C implementation, so adding OpenMP or MPI pragmas around existing loops was straightforward. Debugging CPU code is familiar—print statements, `gdb`, and standard performance profilers (e.g. `perf` or MPI timers) all work out of the box.

Moving to CUDA required learning entirely new concepts:

- Explicit device-memory allocation (`cudaMalloc`/`cudaMemcpy`).

- Writing `__global__` kernels with `threadIdx` and `blockIdx` indexing.

- Managing host–device pointer swaps and CUDA event timing.

- Error-checking after every CUDA call using a macro like `CUDA_CHECK`.

Even once the first CUDA kernel would compile, debugging involved tools such as `cuda-memcheck` or inserting `printf` into kernels, which is much slower and more cumbersome than on the CPU. In practice, getting a correct "toy" CUDA version (e.g. 100×100 grid for a few timesteps) took at least twice as long as turning a serial loop into an OpenMP loop, simply because of unfamiliar syntax and runtime-only errors (out-of-bounds accesses, missing synchronizations, etc.). In short, CPU parallelization (OpenMP/MPI) has a shallow learning curve; CUDA has a significantly steeper one, with more time spent on low-level details before reaching a correct, performant result.

5.2 Performance Gains vs. Programming Cost

On the same cluster node, OpenMP (4–8 threads) and MPI (8–16 processes) delivered up to a 6–7× speedup for large grids, and only modest (or even negative) gains for smaller grids. Writing and tuning the CPU version—choosing loop schedules, avoiding false sharing, or balancing MPI ranks—was relatively quick (hours to a day).

Once the grid size and timestep count grew (e.g. 1000×1000 with 10 000 steps or larger), our CUDA implementation on the A4000 GPU began to pay off. For medium-sized problems, CUDA achieved 15–25× speedup over serial, whereas the best MPI run capped out around 4–6×. That extra 10–20×, however, came at the cost of roughly two to three times more development effort (writing kernels, tuning block sizes, and debugging with specialized GPU tools).

In other words, if your workload is moderate—say, you solve a 1000×1000 grid a few times—MPI or OpenMP gives a "good enough" improvement (factor of 3–5) for very little extra effort. But if your workflow involves many large-grid simulations (2000×2000 or bigger, tens of thousands of steps), then investing the additional weeks of CUDA-specific work can be justified by the order-of-magnitude reduction in runtime.

5.3 Conclusion

From our hands-on experience, implementing a parallel CPU version (OpenMP or MPI) was quick, portable, and easy to debug—yielding respectable speedups for medium- to large-sized problems. By contrast, CUDA programming demanded a steeper learning curve, more intricate debugging, and careful attention to occupancy and memory access. However, once the CUDA kernels were correct and tuned (particularly with 8×8 blocks to maximize occupancy on the A4000), the GPU delivered

a 15–25× speedup on large grids—far outpacing any CPU-only approach. Therefore, unless your application constantly solves very large problems where that extra 10–20× matters, you may prefer the lower development cost of CPU parallelization. But for truly large-scale simulations, the GPU's massive parallelism ultimately pays off in drastically reduced runtimes.‘‘‘